Registration 14th March - 3rd April — Declaration Before R1 — Prepare Now - 25th March — Round 1 25th March - 8th April — Results 10th April — Finale 25th-26th April

ganesh.doosa@gmail.com Copy

Join the Discord Community

All announcements, mentor access, and team matching happens here.

Join Discord

QUICK TOGGLE

Team form Submission | Preparatory Course | Start Assessment | FAQs

---

STEP 1

Choose solo or team before you can start the assessment

STEP 1 COMPLETE

Team: **Agent Smith**

Ganesh Doosa	ganesh.doosa@gmail.com	Team Lead
Gajula Akanksha	gajulaakanksha02@gmail.com	Accepted
Yashwanth Kumar	yashashokgomathi@gmail.com	Accepted

Team is permanently locked. Changes are not allowed after confirmation.

OPENENV ROUND 1 BOOTCAMP OPENENV ROUND 1 BOOTCAMP OPENENV ROUND 1 BOOTCAMP OPENENV ROUND

Live walkthrough to submit a strong Round 1 entry

TIMING

8:00 PM Onwards Wednesday, 1st April

HOST

Ben Burtenshaw
Community Education in AI at Hugging Face

Pulkit Aneja
Scaler Instructor

Watch Recording

PROBLEM STATEMENT

THE TASK

Build a complete, real-world OpenEnv environment that an AI agent can learn from through the standard `step()` / `reset()` / `state()` API.

KEY REQUIREMENTS AT A GLANCE

- Must simulate a real-world task (not games or toys)
- Implement full OpenEnv spec: typed models, step()/reset()/state(), openenv.yaml
- Minimum 3 tasks with agent graders (easy → medium → hard, scores/reward 0.0–1.0)
- Meaningful reward function with partial progress signals
- Baseline inference script with reproducible scores
- Deploy to Hugging Face Spaces + working Dockerfile
- README with environment description, action/observation spaces, setup instructions

Detailed Requirements

FUNCTIONAL REQUIREMENTS

Real-world task simulation

The environment must simulate a task humans actually do. Not games, not toys. Examples: email triage, code review, data cleaning, scheduling, customer support, content moderation.

OpenEnv spec compliance

Implement the full OpenEnv interface: typed Observation, Action, and Reward Pydantic models. step(action) → returns observation, reward, done, info. reset() → returns initial observation. state() → returns current state. openenv.yaml with metadata. Tested via openenv validate.

Minimum 3 tasks with agent graders

Each task defines a concrete objective an agent must accomplish, with a programmatic grader that scores performance (0.0–1.0). Tasks should range: easy → medium → hard. Graders must have clear, deterministic success/failure criteria.

Meaningful reward function

Provides signal over the full trajectory (not just binary end-of-episode). Rewards partial progress toward task co
(e.g. infinite loops, destructive actions).

Submit your Assessment → How to Submit?

Hi! What can I help you with?

Baseline inference script

Uses the OpenAI API client to run a model against the environment. Reads API credentials from environment variables (OPENAI_API_KEY). Produces a reproducible baseline score on all 3 tasks.

NON-FUNCTIONAL REQUIREMENTS

Deploys to a Hugging Face Space

Environment must run as a containerized HF Space tagged with openenv.

Containerized execution

Must include a working Dockerfile. The environment should start cleanly with docker build + docker run.

Documentation

README must include: environment description and motivation, action and observation space definitions, task descriptions with expected difficulty, setup and usage instructions, baseline scores.

Evaluation Criteria

PARAMETER	WEIGHT	DESCRIPTION
Real-world utility	30%	Does the environment model a genuine task? Would someone actually use this to train or evaluate agents?
Task & grader quality	25%	Are tasks well-defined with clear objectives? Do graders accurately and fairly measure success? Meaningful difficulty progression?
Environment design	20%	Clean state management, sensible action/observation spaces, good reward shaping, proper episode boundaries.
Code quality & spec compliance	15%	Follows OpenEnv spec, clean project structure, typed models, documented, tested, Dockerfile works.
Creativity & novelty	10%	Novel problem domain, interesting mechanics, clever reward design, original approach.

SCORING BREAKDOWN

- 0–5: Toy/artificial problem with no practical application
- 6–15: Valid domain but shallow modeling of the real task
- 16–25: Good domain modeling, would be useful for agent evaluation

- 3+ tasks with difficulty range?
- Graders produce scores between 0.0–1.0?
- Graders deterministic and reproducible?
- Hard task genuinely challenges frontier models?

- reset() produces clean state?
- Action/observation types well-designed and documented?
- Reward function provides useful varying signal (not just sparse)?
- Episode boundaries sensible?

- openenv validate passes?
- docker build && docker run works?
- HF Space deploys and responds?
- Baseline script runs and reproduces scores?

- Domain we haven't seen in OpenEnv before?
- Reward design has interesting properties?
- Clever mechanics that make the environment engaging?

How Judging works

Pass/fail gate — HF Space deploys, OpenEnv spec compliance, Dockerfile builds, baseline reproduces, 3+ tasks with graders.

Scored — baseline agent re-run, standard Open LLM agent (e.g. Nemotron 3 Super) run against all environments, score variance check.

Top submissions reviewed by Meta and Hugging Face engineers for real-world utility, creativity, and exploit checks.

DISQUALIFICATION CRITERIA

- ⊗ Environment does not deploy or respond
- ⊗ Plagiarized or trivially modified existing environments
- ⊗ Graders that always return the same score
- ⊗ No baseline inference script

Pre-Submission Checklist — all must pass or you're disqualified

HF Space deploys	Automated ping to the Space URL — must return 200 and respond to reset()
OpenEnv spec compliance	Validate openenv.yaml, typed models, step()/reset()/state() endpoints
Dockerfile builds	Automated docker build on the submitted repo
Baseline reproduces	Run the submitted inference script — must complete without error and produce scores
3+ tasks with graders	Enumerate tasks, run each grader, verify scores/reward in 0.0–1.0 range
Mandatory Additional Instructions	Before submitting, ensure the following variables are defined in your environment configuration: API_BASE_URL The API endpoint for the LLM. MODEL_NAME The model identifier to use for inference. HF_TOKEN Your Hugging Face / API key. The inference script must be named `inference.py` and placed in the root directory of the project Participants must use OpenAI Client for all LLM calls using above variables Participants must emit structured stdout logs strictly following the [START], [STEP], and [END] format defined in the sample inference.py provided below. Any deviation in field names, ordering, or formatting will result in incorrect evaluation scoring. Refer to the Sample Inference Script for the complete format specification and examples.
Infra Restrictions	Runtime of inference script should be less than 20min Make sure your env and inference can run on a machine with vcpu=2, memory=8gb

How to Submit?

Hi! What can I help you with?

Validator — Run the pre-submission validation script before submitting

NEW

NEW

Submission window opens on 28th March

Deadline: 8 Apr 11:59 PM

Submit your Assessment →

STUDY MATERIAL

4 modules · ~3.5 hours

Each module: read the README first, then open the notebook in Colab. No local setup needed.

ESSENTIAL FOR ROUND 1	45 min
ESSENTIAL FOR ROUND 1	50 min
ESSENTIAL FOR ROUND 1	45 min
MOST IMPORTANT FOR ROUND 1	60 min

View full course repository

GUIDE

What to Expect | Prerequisites | How to Submit

Example of what a problem statement looks like

"Build a mini-game RL environment with clearly defined tasks, automated graders, and reward logic using the OpenEnv framework."

→ Create a mini-game an AI agent can play
→ Define tasks with increasing difficulty
→ Write graders that verify task completion
→ Define reward logic for scoring
→ Package using OpenEnv for automated evaluation

Evaluation Criteria

Runtime correctness	Runs without errors
Interface compliance	Follows OpenEnv standard
Task design	Clear, realistic, testable
Grading logic	Reward system makes sense

STEP 2

Complete Step 1 first

Problem Statement is live. Build and submit.

Submission window opens on 28th March

Deadline: 8 Apr 11:59 PM

Submit your Assessment →

FAQS

How does the team/solo declaration work?

Who should fill the team form?

What if someone already added me to their team?

Can I change my team or switch to solo after confirming?

Do I need to complete the prep course?

What happens during Round 1?

Hi! What can I help you with?

Can I update my submission?

How are submissions evaluated?

What framework must be used?

What happens after Round 1?

What do I need to submit?

Where can I get help?

Need help? Reach out to us

help_openenvhackathon@scaler.com

Contact Support

Hi! What can I help you with?